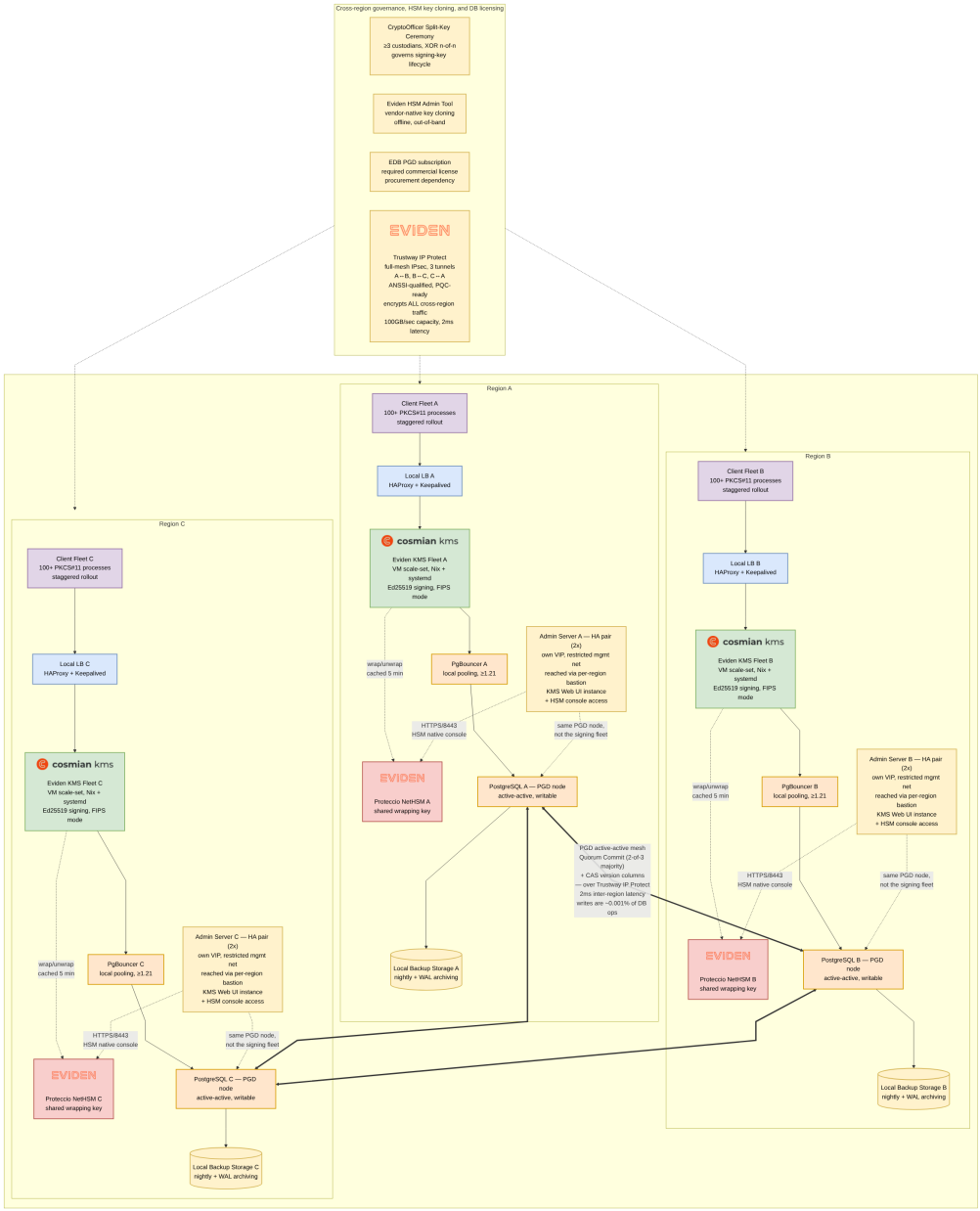
Eviden KMS — 3-Region Signing Cluster: Architecture Report

Session date: 2026-09-04 **Goal:** 3-node/3-region Cosmian KMS cluster, fully redundant key dataset (survives losing one node/region), keys encrypted by a per-region HSM under a shared wrapping key, used for high-throughput data signing.

This report consolidates a 29-decision interview conducted against the actual KMS codebase (not assumptions) — each decision below cites the code/doc that grounded it.

---

0. Global Architecture Diagram

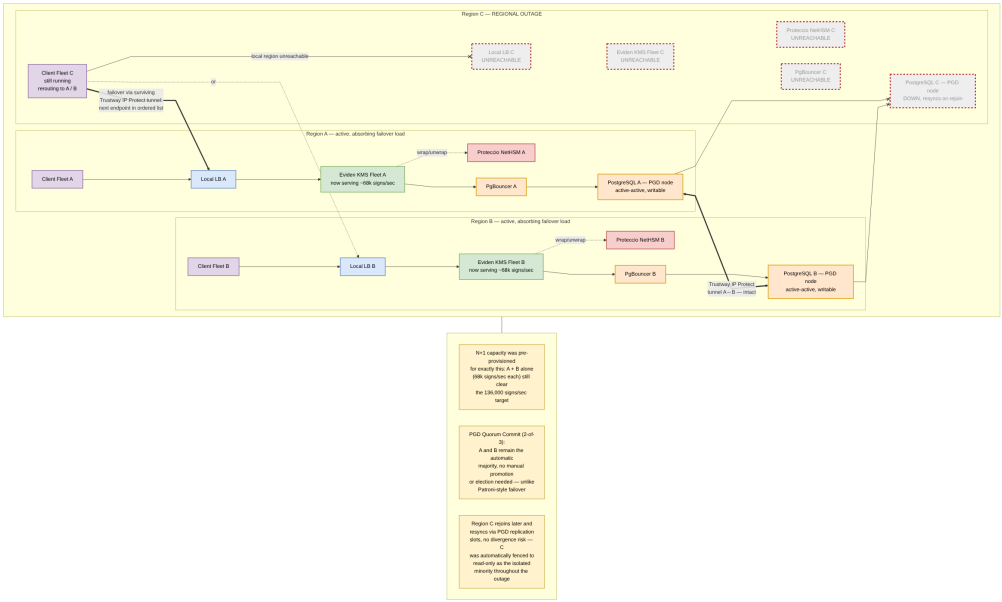


Global architecture diagram

Color legend: purple = client fleet (PKCS#11 processes), blue = local load balancer, green = Eviden KMS fleet, orange = PostgreSQL/PGD/PgBouncer, red = Proteccio HSM, yellow = cross-cutting governance/ops/licensing/backup/network. Dotted edges are out-of-band or cache-bounded relationships (HSM wrap/unwrap, ceremony/cloning/licensing governance); the thick edges are the PGD active-active replication mesh, itself carried over a full-mesh **Trustway IP Protect** IPsec tunnel (3 tunnels: A↔B, B↔C, C↔A) that encrypts *all* cross-region traffic — PGD replication and client failover alike; thin solid edges are the live request path.

Logos: Cosmian and Eviden marks are this repository's own assets (`ui/public/themes/cosmian/`, `documentation/docs/eviden-logo-orange.svg`), used here for the same product they label in-repo. PostgreSQL, Ubuntu, and EDB logos are intentionally **not** included — this sandbox has no general internet access for binary image downloads (confirmed via both raw `curl` and the `web_fetch` tool), and no repo-included copy of those third-party marks exists to reuse. Rather than reconstruct trademarked logos from memory, those components are labeled by name only.

0b. Failure Scenario: Losing One Region



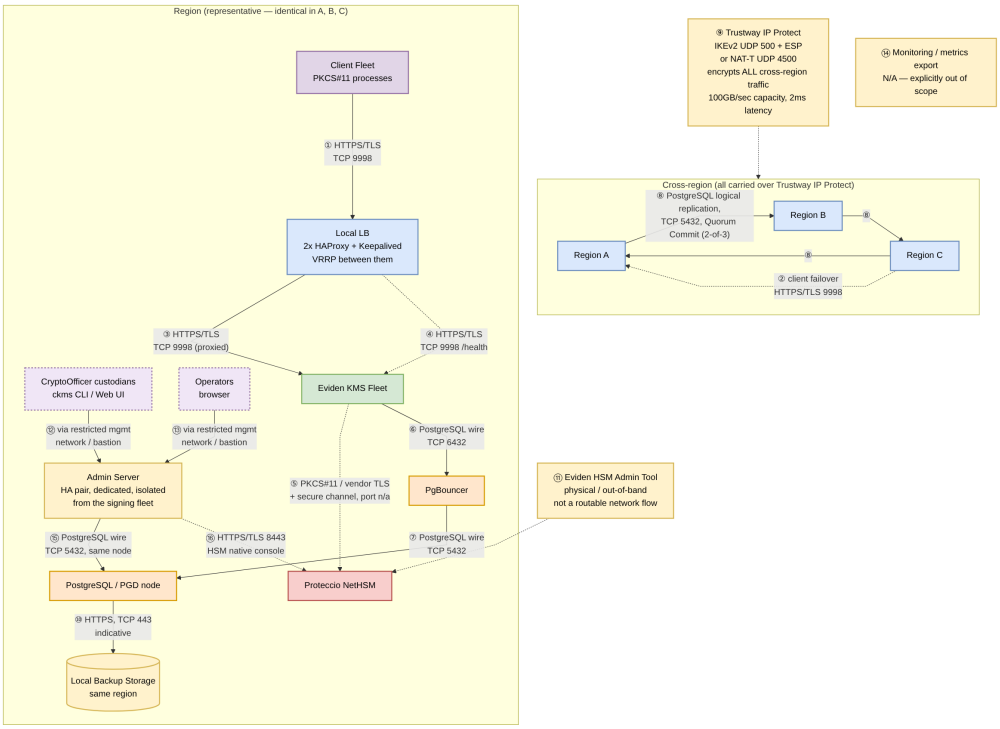
Failure scenario diagram: Region C outage

Region C (grey, dashed red border) suffers a full regional outage — client fleet included, its local KMS/HSM/PgBouncer/PostgreSQL are all unreachable. What the architecture guarantees in this state, per the decisions already made:

- **No manual promotion, no interruption**: PGD's **Quorum Commit** (2-of-3 majority — see §4) keeps Regions A and B as the automatic majority partition; they simply continue as active-active peers the moment C drops off the mesh (shown as the red × links from `pgA`/`pgB` to `pgC`) — no election or manual failover step, unlike Patroni-style promotion.
- **Client failover is app-level, not infra-level**: Region C's own clients reroute using the ordered-endpoint-list + retry policy (§1) to Region A or B's local load balancer — no DNS change, no anycast, no manual intervention.
- **Capacity was sized for exactly this**: the N+1 provisioning decision (§6, 204,000 signs/sec total, 68,000/sec per region) means A and B absorbing all of C's traffic together deliver exactly the 136,000 signs/sec failover target.
- **Recovery is safe by construction**: when Region C rejoins, it resyncs via PGD's replication slots. Because C was the isolated minority throughout the outage, Quorum Commit automatically fenced it to **read-only** the moment it lost contact with the majority — it cannot have diverged, so it catches up cleanly rather than reconciling a conflicting history. (Writes are a vanishingly

small fraction of traffic in the first place — confirmed this session at ~0.001% of all DB operations, §4 — so this fencing has no practical throughput cost.)

0c. Network Flow Matrix



Network flow diagram, numbered to match the matrix below

For firewall rule configuration and security review. Ports are cited from verified sources: KMS defaults from `cosmian_kms --help` (this session), PostgreSQL/PgBouncer from their own project defaults, IPsec ports from the IETF standard. Where a port isn't exposed by this repo's config (Proteccio, backup storage), that's stated explicitly rather than invented.

#	Source	Destination	Protocol / Port	Usage
1	Client Fleet (PKCS#11, local region)	Local LB (same region)	HTTPS/TLS, TCP 9998 (KMIP JSON-TTLV over HTTP — confirmed this is the path `cosmian_kms_client::KmsClient` uses, **not** the raw KMIP socket server on 5696, which this architecture doesn't use)	`Sign` hot path (136,000 signs/sec failover target, §6)
2	Client Fleet C (during regional outage)	Local LB A or B (cross-region)	HTTPS/TLS, TCP 9998, carried over the Trustway IP Protect tunnel	App-level failover per ordered endpoint list (§0b)
3	Local LB	KMS Fleet (same region, backend)	HTTPS/TLS, TCP 9998 (proxied)	Load-balanced application traffic
4	Local LB	KMS Fleet (same region)	HTTPS/TLS, TCP 9998, `GET /health`	HAProxy liveness probe (documented pattern, `high_availability_mode.md`) — same port/protocol as row 3: the KMS server has no separate cleartext health-check path, `/health` is served under the same TLS configuration as the rest of the app

#	Source	Destination	Protocol / Port	Usage
5	KMS Fleet	Proteccio NetHSM (same region)	PKCS#11 API over the vendor's own TLS + EC-based secure channel; **port is vendor-specific** — configured in `/etc/proteccio/proteccio.rc` (`IPaddr=`), not exposed via KMS config	Key wrap/unwrap (cached 5 min per §6), HSM admin ops
6	KMS Fleet	PgBouncer (same region, local)	PostgreSQL wire protocol, TCP 6432 (PgBouncer project default — not verified in this repo's docs)	Key-lifecycle DB reads/writes (`Sign` itself has zero DB writes — §6)
7	PgBouncer	PostgreSQL/ PGD node (same region, local)	PostgreSQL wire protocol, TCP 5432 (confirmed default, `configuration.md`)	Pooled connection to the always-local-writable PGD node
8	PostgreSQL/ PGD node A ↔ B ↔ C	(cross-region mesh)	PostgreSQL logical replication protocol, TCP 5432, carried over the Trustway IP Protect tunnel — **2ms inter-region latency** (per-hop, one-way/RTT not specified)	Active-active replication; **Quorum Commit (2-of-3 majority)** on the `objects` table (§4, refined from an initial eager/global choice) — adds ~1 round trip (≈2–4ms) to key-lifecycle writes only, never to `Sign`. Writes are ≈0.001% of all DB operations (confirmed this session), so this cost is negligible in practice
9	Trustway IP Protect tunnel endpoints, A ↔ B ↔ C	(network-layer, underlies rows 2 and 8)	IPsec: IKEv2 (UDP 500) + ESP (IP protocol 50), or NAT-T (UDP 4500) — standard IETF IPsec ports. **Capacity: 100GB/sec** (as stated; if this means 100 Gbps, utilization at the 136,000 signs/sec failover target is ≈2.2%; if literally 100 gigabytes/sec, ≈0.28% — negligible either way, see §1)	Encrypts **all** cross-region traffic (per the earlier "all-cross-region" scope decision)
10	PostgreSQL/ PGD node	**Local** Backup Storage (same region — corrected from an earlier shared/ centralized design, §5)	HTTPS, TCP 443 (typical S3-compatible object storage API — the specific backup tool wasn't chosen this session, so this is indicative, not verified)	Nightly backup + WAL archiving, quarterly restore drills (§5)
11	Eviden HSM Admin Tool	Proteccio NetHSM (all 3 regions)	Vendor-specific administrative protocol; smartcard/PED-based — largely **physical/out-of-band**, not a routable network flow	One-time cross-site wrapping-key cloning ceremony (§3)
12	CryptoOfficer custodians (via `ckms` CLI or Web UI)	Admin Server VIP (own region, HA pair)	HTTPS/TLS, TCP 9998 — **only reachable via the region's restricted management network / bastion**, not the general corporate network	Split-key ceremony ops (`CreateSplitKey`/`JoinSplitKey`), key lifecycle (§2)
13	Operators (browser)	Admin Server VIP (own region, HA pair)	HTTPS/TLS, TCP 9998, Web UI — same bastion-only restriction as row 12	Admin console: HSM status (`GET /hsm/status`), ceremony stamping
15	Admin Server (own region)	PostgreSQL/ PGD node (same region, local)	PostgreSQL wire protocol, TCP 5432	Dedicated admin-plane KMS instance reads/writes the same live PGD data — **not** part of the load-balanced signing fleet (rows 1/3/4), isolating admin/management traffic from the signing hot path
16	Admin Server (own region)	Proteccio NetHSM (same region)	HTTPS, TCP 8443 (vendor-documented native admin console — verified)	HSM-native administration: virtual HSM management, SO/CO roles and PINs, firmware, security policy
14		—	N/A	

#	Source	Destination	Protocol / Port	Usage
	Monitoring/ metrics export			**Explicitly out of scope** — handled by an existing external system, not this repo's stack (§1)

> **Note — Row 4 abstraction.** Rows 1–4 model each region's "Local LB" as one logical box, matching the diagrams in §0/§0b. A real deployment following `documentation/docs/installation/high_availability_mode.md` would run this as **2** physical HAProxy+Keepalived nodes per region, with a Keepalived VRRP flow (IP protocol 112, multicast) between them — omitted here for consistency with the diagrams, but worth remembering when translating this matrix into firewall rules.

> **Note — Rows 12/13/15/16, Admin Server topology.** The Admin Server is itself an HA pair (2 nodes, its own dedicated Keepalived VIP — separate from the signing fleet's LB, so the fleet's LB nodes are never a network path to HSM Security-Officer-level access) per region, reachable only via that region's own bastion/jump-host — deliberately per-region, not centralized, so losing one region's admin plane never blocks CryptoOfficer ceremonies or HSM administration in the other 2.

---

1. Topology & Deployment

Decision	Answer
Cluster shape	3 geographic regions, **all 3 active-active/primary** (see §4 — revised mid-session from an initial single-primary assumption) + 1 local Trustway Proteccio HSM per region, sharing one logical multi-master-replicated dataset
Packaging	**Nix-built `.deb` + systemd**, target OS confirmed **Ubuntu Server** (not Kubernetes). Corrected from an earlier generic "deb/rpm" — rpm doesn't apply on a Debian-family OS. Built via `mise run package:deb` (`.mise/tasks/package/deb`, verified in-repo), which wraps `package_common.sh --format deb --variant fips --link static`
Scaling mechanism	Cloud VM scale-set / instance-group running the Nix-built systemd service, custom scaling policy — reconciles "systemd" with "need autoscaling" (K8s+Helm+HPA was the alternative, rejected)
Client routing across regions	App-level ordered endpoint list + retry (no DNS georouting/GSLB/anycast infra) — simplified further by §4's pivot: clients no longer need to discover "the" primary, since every region is always locally writable
Inter-region network transport	**Eviden Trustway IP Protect** — full-mesh IPsec (3 tunnels: A↔B, B↔C, C↔A). ANSSI-qualified, post-quantum-ready (verified: same "Trustway" product family as the Proteccio HSM already in this architecture). Carries **all** cross-region traffic — PGD replication and client failover alike — so no cross-region traffic is ever exposed on the public network layer. **Capacity: 100GB/sec** (as stated — dramatically more than the workload needs, see calculation below). **Latency: 2ms** inter-region — adds ~1 round trip to key-lifecycle writes only (PGD Quorum Commit, §4); `Sign` never crosses regions in steady state so is unaffected
Admin/management plane	**Dedicated Admin Server per region, as an HA pair** (2 nodes, own dedicated Keepalived VIP — deliberately *not* shared with the signing fleet's LB, so the fleet's LB nodes are never a network path to HSM Security-Officer-level

Decision	Answer
	access) — runs its own KMS instance (Web UI) against the same regional PGD node, plus network access to the Proteccio NetHSM's own native web console (verified: HTTPS/8443). Reachable only via a **per-region bastion/ restricted management network** (not the general corporate network) — deliberately per-region so losing one region's admin plane never blocks ceremonies or HSM administration elsewhere. CryptoOfficer ceremony ops and Operator Web UI access both route here, not through the Fleet's load balancer — reduces attack surface on the >136,000 signs/sec signing path
Per-site backup	Each region backs up to its **own local** encrypted storage (not shared/centralized) — see §5 for the full rationale and trade-off
Monitoring	Handled by an existing external system — **not** wired into this repo's Prometheus/Grafana/OTLP stack

Bandwidth sanity check: at the 136,000 signs/sec failover target, even generously assuming a 2KB KMIP request+response per `Sign` call, peak client-failover traffic is only **≈2.2 Gbps** — about 2.2% of a 100 Gbps link, or 0.28% if "100GB/sec" is meant literally (800 Gbps). PGD replication traffic is smaller still (10 keys, rare lifecycle writes only). **Cross-region bandwidth has enormous headroom for this workload regardless of which reading is correct** — it will not be a bottleneck.

Key fact: **Cosmian KMS is stateless.** Its only documented HA pattern (`documentation/docs/installation/high_availability_mode.md`) is "shared DB + load balancer." There is no built-in multi-region replication or HSM key-sync — this entire architecture lives in the DB replication topology, HSM vendor tooling, and client routing layers.

---

2. Cryptography & Compliance

Decision	Answer
FIPS 140-3	Required
Signing algorithm	**EdDSA Ed25519** (KMIP `Sign`) — FIPS-approved per `documentation/docs/certifications_and_compliance/cryptographic_algorithms/algorithms.md`
HSM wrapping enforcement	`key_encryption_key` forced globally so every signing key is created already HSM-wrapped
Unwrapped-key cache exposure window	`unwrapped_cache_max_age = 5 min`
Key rotation	Quarterly, using the existing keyset-chain auto-rotation (ADR-0002, `2026-06-30-key-auto-rotation-keyset-chain-design.md`) — old signatures stay verifiable after rotation; HSM-resident keys rotate via `C_GenerateKey` inside Proteccio, no software extraction
Governance	CryptoOfficer split-key ceremony **enabled**, ≥3 custodians, XOR n-of-n (`documentation/docs/configuration/authorization/key_ceremony.md`) — no single admin can unilaterally export/destroy/rekey a signing key

---

3. HSM

Decision	Answer
Vendor	Trustway Proteccio (native Eviden integration, `crate/hsm/proteccio`)

Decision	Answer
Cross-site key cloning	✅ **RESOLVED.** The KMS references HSM keys as `hsm::<model>::<slot>::<label>` per-node; it has **no code path** to copy key material between physically separate HSMs — this is by design, entirely outside the KMS. Confirmed: the shared wrapping key will be cloned into all 3 regional Proteccio appliances using the **Eviden HSM admin tool** (vendor-native backup/cloning ceremony, smartcard/PED-based), independent of KMS configuration.

---

4. Database & Replication — ⚠️ major revision: single-writer → true active-active multi-master

This section was completely revised mid-session. The original plan (1 primary + 2 sync standbys, Patroni + etcd failover) assumed exactly one writable region at a time. The user corrected this: **all 3 regions must be primary — true active-active, concurrent writes to the same key objects from any region.**

This is a fundamental conflict with what was originally chosen, confirmed by direct inspection of the codebase (not assumption):

- `documentation/docs/configuration/database/configuration.md` documents `target_session_attrs=read-write` — the KMS's own driver logic is built to find **the one** currently-writable node and skip standbys. This assumes single-writer.
- Patroni's entire purpose is electing exactly **one** primary at a time.
- A repo-wide search for "multi-master", "active-active", "BDR" returned **zero hits** — no code path in this KMS anywhere anticipates concurrent regional writes.
- **Worse:** `crate/server_database/src/stores/sql/query.sql` defines the `objects` table with **no version column, no `updated_at`, no CAS guard whatsoever** — every `UPDATE` query (`update-object-with-object`, `update-object-with-state`, `update-wrapping-key-id`) matches only on `id` and blindly overwrites. Two regions concurrently mutating the same key (e.g. region A runs `Destroy` while region C runs `ReKey` on the same key ID) would silently resolve however the replication engine's default conflict policy picks, with the KMS application layer completely unaware a conflict occurred.

Resolved decisions

Decision	Answer
Engine	**EDB Postgres Distributed (PGD/BDR)** — logical multi-master replication, every regional node independently writable, no external DCS needed
Failover orchestrator	**None — removed.** Confirmed via EDB documentation: PGD needs no Patroni/etcd/Consul; every node is a full peer with "zero-wait failover" built into the extension itself
Conflict-window closure	**Quorum Commit (2-of-3 majority)** for the `objects` table — refined from an initial "Eager/global" (`bdr.commit_scope='global'`, all 3 nodes) choice once a second, distinct PGD commit-scope option was found and verified. Eager/global would have halted writes **cluster-wide** during any partition, even a transient blip affecting only 1 region; Quorum Commit lets the 2-region majority keep operating normally while automatically fencing the isolated minority to read-only — see the Partition/fencing resolution below
Conflict detection (defense-in-depth)	**Add version/CAS columns** to the `objects` table and update all write paths in `crate/server_database`.

Decision	Answer
	Conflict handling resolved: on a detected stale write (0 rows affected), the KMIP operation handler returns an explicit conflict error, forcing the client to re-fetch and retry — no silent server-side auto-retry, since that would be unsafe for destructive operations like `Destroy` racing a concurrent `ReKey`
Connection pooling	PgBouncer retained per region, but **simplified**: each region's PgBouncer now points to its **own local** PGD node only — the old 3-host `target_session_attrs=read-write` failover string is no longer needed, since every region is always locally writable. **EDB's PGD Proxy was evaluated and explicitly not adopted** — see below
Positive side effect	This resolves the earlier §6 concern (non-primary regions paying WAN latency on every cache miss) — there is no longer a "non-primary region"; every region always has a local writable DB
Workload characterization	Confirmed this session: **writes are ≈0.001% of all database operations** (99.999% reads) — reinforces that Quorum Commit's fencing overhead, the CAS conflict-handling design, and skipping PGD Proxy are all low-risk choices, since the scenario they govern (concurrent writes to the same key) is already an exceptionally rare event class

⚠️ New procurement dependency

EDB Postgres Distributed requires a paid EDB commercial subscription — confirmed there is no free/open-source PGD tier (only vanilla community PostgreSQL is free; the active-active distributed features are gated behind EDB's subscription). This is a licensing/procurement dependency of the same category as the Proteccio HSM cloning ceremony (§3) — **must be resolved with EDB before this architecture is buildable.**

Partition/fencing behavior — resolved

Previously flagged as open: if a region is fully network-isolated (not just "down"), what stops prolonged divergence before it can rejoin and reconcile? **Resolved by the Quorum Commit choice above.** Verified via EDB documentation: PGD's Quorum Commit mode detects when a node/group loses contact with the majority and **automatically fences it to read-only** — no custom tooling, no manual intervention, and no risk of the isolated minority accepting conflicting writes. This is exactly what the §0b failure scenario now describes for Region C.

PGD Proxy — evaluated and not adopted

EDB's PGD Proxy (a dedicated connection-routing component) was investigated as a candidate for automatically rerouting away from a fenced node. **Verified finding that changed the recommendation**: PGD Proxy's routing model elects a single **write leader** per node group and routes all writes through it at a time (with automatic failover on leader loss) — architecturally much closer to fast-failover single-writer than the **true active-active, all-3-regions- simultaneously-writable** model explicitly required earlier in this session. Adopting it as typically configured would have silently reverted that decision. **Decision: skip PGD Proxy.** Each region's Fleet keeps writing directly to its own local PGD node via PgBouncer; if that node is fenced during a partition, key-lifecycle writes fail and are handled at the application/retry level (never `Sign`, which has no DB write) rather than being silently rerouted to another region's node.

---

5. Backup/DR

Nightly encrypted backups + WAL archiving + **quarterly restore drills**, independent of live replication — because replication faithfully propagates logical mistakes too (accidental `Destroy`, bad migration, ransomware). This is unaffected by the PGD pivot: each regional PGD node still needs independent physical/logical backups.

Backup storage is per-site (local to each region), not centralized. Each region's PGD node backs up to its own local encrypted storage (§0 diagram: `backupA`/`backupB`/`backupC`), rather than all 3 regions shipping backups to one shared destination. This was a deliberate correction from the original design (which had a single shared backup sink reachable via `regions --> backup`):

- **No cross-region dependency for backup/restore.** A region's nightly backup and restore drill don't rely on the Trustway IP Protect tunnel or another region being reachable — relevant since that tunnel is also carrying the PGD replication mesh and client failover traffic (§0c); backup shouldn't compete with or depend on that same path.
- **No single point of failure for the backup function itself.** A shared centralized store would have been a 4th site whose loss affects all 3 regions' DR posture simultaneously.
- **Consistent with the "site" framing already used elsewhere** in this architecture (per-site HSM, per-site PGD node, per-site PgBouncer) — backup is naturally another per-site component, not an exception.
- **Trade-off, stated plainly:** if a region is destroyed entirely (not just down), its own local backup is destroyed with it. This is an accepted trade-off given PGD's active-active replication already keeps a live, current copy of the data on the *other* 2 regions — local backups protect against **logical** mistakes (accidental `Destroy`, bad migration, ransomware propagating through the replication mesh), not **physical site loss**, which the multi-region replication itself already covers. If protection against a specific region's backups being lost along with the region is also wanted, that requires a 4th, independent cross-region backup replication target — not decided here; flagged as an open question rather than assumed.

---

6. Performance & Scalability — the major mid-session findings

This branch produced the most consequential discoveries and two corrections:

- **Throughput target corrected:** originally ">10,000 signs/sec" → corrected to ">100,000 signs/sec" → **pinned precisely to 136,000 signs/sec**, confirmed as the load that must be sustained even with **1 region down** (the N+1 failover target, i.e. the combined output of the 2 surviving regions).
- **N+1 capacity:** 136,000 signs/sec ÷ 2 surviving regions ⇒ **68,000 signs/sec per region**, provisioned at all times (not just during failure) ⇒ **204,000 signs/sec total** 3-region steady-state capacity (68,000 × 3).
- **Key cardinality:** confirmed **exactly 10** signing keys in production use ⇒ `ObjectCache`/`UnwrappedCache` (ADR-0003, moka-based, default 1000-entry capacity) stay trivially fully warm — 1% of capacity, zero LRU-thrashing risk; HSM/DB round-trips happen once per 5-min cache window **per key**, not per signing call — confirmed `crate/server/src/core/operations/sign.rs` has **no DB write** on the hot path. **This is also why the PGD Quorum Commit scope (§4) is nearly free**: it only adds latency to key-lifecycle mutations (Create/ReKey/Destroy/SetAttribute), never to the `Sign` hot path.
- **Client connection model:** keep-alive/pooled connections (best case for TLS overhead).
- **Rollout policy:** staggered instance start-up for deploys and scale-out events, to avoid a cold-cache stampede against one region's Proteccio appliance + Postgres primary.
- **Benchmark gap closed (partially):** the repo's own `cpu_scaling.md` benchmark only covered AES-GCM / RSA-OAEP / ECDSA-P256 in **non-FIPS** mode. Added an `eddsa_sign` case to `crate/test_kms_server/benches/http_throughput.rs` (compiles, passes `cargo clippy -D warnings`).

⚠️ Critical correction: client protocol is PKCS#11, not KMIP/REST

Mid-session the user corrected the assumed client integration: signing clients talk to the KMS via the **PKCS#11 interface** (`crate/clients/pkcs11/provider`, the `cosmian_pkcs11` shared library — built for LUKS/Veracrypt/Prim'x-style disk-encryption integrations), not a direct KMIP/HTTP client.

Tracing the call path (`crate/clients/pkcs11/provider/src/backend.rs` → `crate/clients/pkcs11/provider/src/kms_object.rs::kms_sign` → `cosmian_kms_client::KmsClient`) confirmed the module still ultimately performs the **same** KMIP `Sign` HTTP call the benchmark exercises — so the server-side cache/HSM analysis above still holds. But it surfaced a **severe, concrete bottleneck**:

> **`crate/clients/pkcs11/module/src/sessions.rs:652-662`** — the `session()` helper acquires a single **global, process-wide `Mutex<HashMap<CK_SESSION_HANDLE, Session>>`**, then invokes the operation callback **while still holding that lock**. For `C_Sign`, the callback calls `session.sign()` → `RUNTIME.block_on(kms_sign_async(...))` — a **blocking network round-trip to the KMS server** — all inside the locked section.
>
> **Effect:** every PKCS#11 operation (Sign, Encrypt, Decrypt, FindObjects) across **every session in one process** is fully serialized on the network round-trip. One loaded module instance ≈ **1 signing request in flight at a time**, regardless of how many threads the calling application spawns. Throughput per process ≈ `1 / round-trip-latency` — at a plausible ~1–2 ms round-trip that's only ~500–1,000 signs/sec **per process**.

Decision: accept this operationally rather than patch the lock — provision **100+ independent OS processes**, each loading the PKCS#11 module, to reach 136,000 signs/sec in aggregate. (The alternative — releasing the lock before the blocking call, mirroring the pattern already used in `crate/hsm/base_hsm/src/base_hsm.rs::get_slot()` — was offered but declined.)

Follow-on caveat found: `crate/server/src/middlewares/rate_limiter.rs` rate-limits **per source IP** (`governor` crate), disabled by default (`rate_limit_per_second: None`). If enabled later for DoS protection, the quota must be sized for the *aggregate* throughput of every PKCS#11 process sharing a host's IP, not a single client.

6b. Benchmark: Signing Performance

Two different things need measuring for this architecture, and they answer two different sizing questions — this section covers the first; §7 covers the second (and remains blocked):

Measures	Sizes	Status
Server-side throughput under HTTP concurrency (this section)	How many **KMS server instances** per region (§6 N+1 capacity)	✅ measured this session
Single-request sequential latency (§7)	How many **PKCS#11 client processes** per region (§6's "100+ processes" finding)	⚠️ still blocked — no FIPS provider in this sandbox

What was run

Added an `eddsa_sign` case to the repo's own `crate/test_kms_server/benches/http_throughput.rs` Criterion bench (alongside the pre-existing `ecdsa_sign`, `aes_gcm_enc`, `rsa_oaep_dec` cases — see `documentation/docs/benchmarks/cpu_scaling.md`), then ran it for real:

```
cargo bench --bench http_throughput -p test_kms_server --features non-fips -- sign
```

This drives 16 concurrent async KMIP `Sign` requests per Criterion iteration against an in-process KMS (SQLite on tmpfs), sweeping `server_workers ∈ {1, 2, 4, 8}` — i.e. it measures **server-side capacity**, not one client's achievable rate.

Results (this session, measured — not from the historical docs)

Environment	Value
Date	2026-09-07
Build	`bench` profile / **non-FIPS** (this sandbox has no `fips.so` — see §7)
CPU	12th Gen Intel(R) Core(TM) i7-1265U, 12 logical CPUs
RAM	31 GB
Database	SQLite (tmpfs), single run

Workers	ecdsa_sign (req/s)	eddsa_sign (req/s)	Ed25519 speedup
w1	2,249	5,875	2.6×
w2	3,544	9,259	2.6×
w4	5,787	11,479	2.0×
w8	7,273	13,455	1.8×

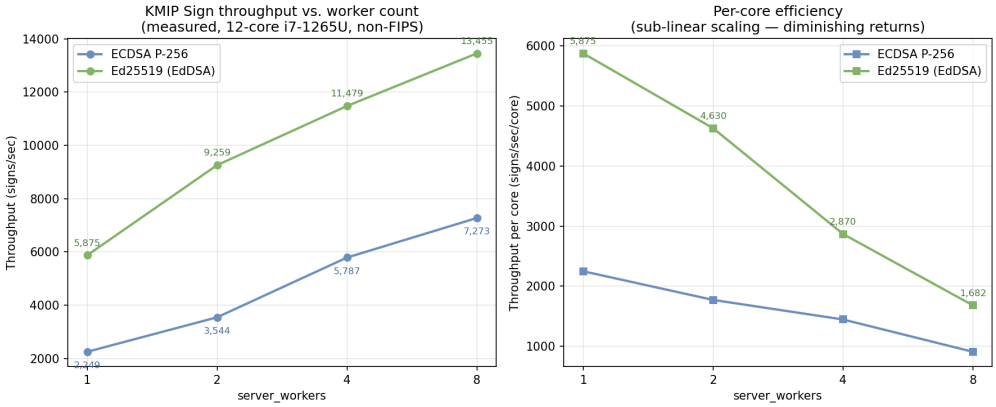
Ed25519 (the algorithm chosen in §2) is consistently **1.8×–2.6× faster** than ECDSA P-256 at every worker count on this hardware — consistent with the repo's own pure-crypto-primitive numbers in `documentation/docs/benchmarks/ckms_bench/report.md` (`eddsa-ed25519/sign` = 87.0 µs vs `ecdsa-p256/sign` = 263.6 µs, ≈3× at the primitive level; the smaller 1.8×–2× gap at higher worker counts reflects HTTP/actix/Tokio dispatch overhead increasingly dominating as concurrency rises).

> **Warning — not directly comparable to the repo's published `cpu_scaling.md` numbers.** That doc's ECDSA P-256 numbers (w1=4,962 → w8=13,805 req/s) were measured on a 24-core/32-thread Intel i9-14900T. This session's numbers were measured on a 12-core i7-1265U — roughly half the cores, and this session's ECDSA figures are correspondingly lower at every worker count. The **relative** Ed25519-vs-ECDSA ratio is the reusable finding; the **absolute** req/s ceiling should be re-measured on production-class hardware before committing to a node count.

What this does and doesn't tell us

- **Confirms** the §6 finding that `Sign` has no DB write on the hot path: throughput scales with `server_workers` (more OS threads → more concurrent OpenSSL operations), not with database I/O.
- **Does not** measure FIPS-mode overhead — this sandbox lacks the OpenSSL FIPS provider (`fips.so`); the FIPS provider's additional per-context locking (noted in `cpu_scaling.md`) means real production numbers will be somewhat lower than these non-FIPS figures.
- **Does not** tell us the PKCS#11 per-process ceiling — that is bounded by the single-request round-trip latency and the global session mutex (§6's critical finding), which is a client-side constraint this server-throughput benchmark cannot measure. That number is still §7's open item.

Scaling curve and core-sizing implications



Sign throughput scaling: absolute and per-core efficiency

The right panel is the important one. Per-core throughput **drops by roughly half with each doubling of worker count** (marginal contribution of each added core, Ed25519): +3,384 req/s for the 2nd core, +1,110 req/s/core for cores 3–4, +494 req/s/core for cores 5–8. This is a much stronger and more consequential signal than the left panel's "throughput keeps climbing" line suggests.

Naive extrapolation to a 136,000 signs/sec target (confirmed as the precise N+1 failover target — the combined output required from the 2 surviving regions, §6) gives wildly different answers depending which point on this curve you extrapolate from:

Method	Assumption	Cores implied
Best single-core rate (w1)	No contention, ever — clearly false	~23
Average rate at w8	Efficiency plateaus at the w8 level	~81
Marginal rate, w4→w8	Contention keeps worsening at the same pace	~256

The real risk this curve exposes: if the halving pattern continues, marginal per-core throughput approaches zero well before 136,000 signs/sec — meaning **a single KMS process might never reach this target no matter how many cores it's given**, if the bottleneck is architectural (single Tokio runtime, single `moka` cache instance, single listener socket) rather than raw CPU availability. Adding cores to one giant process is not a safe assumption here.

This validates rather than changes the architecture already chosen (§1's VM scale-set, §6's "100+ PKCS#11 processes"): horizontal scaling — many modestly-sized processes — sidesteps this single-process ceiling entirely, and was already required anyway by the client-side session lock. Picking a per-process worker count near the knee of the curve (e.g. **w4 = 11,479 req/s**, a reasonable balance of absolute throughput and per-core efficiency) and scaling **process count**, not core count, gives: `[136,000 / 11,479] = 12 processes` — plausibly *fewer* total cores than any of the single-giant-process estimates above, because it avoids whatever contention causes the sub-linear curve in the first place.

To get a trustworthy number instead of this reasoned bound: rerun this same benchmark with `server_workers` up to 16/32/64 on server-grade (uniform-core, non-hybrid) hardware matching the actual deployment target — this sandbox's 12-thread hybrid mobile CPU (2 performance cores + 8 efficiency cores) cannot be pushed further and is not representative of a real deployment target in the first place.

⚠️ Correction: the criterion benchmark methodology understated real throughput by ~2–3×

Everything above this point was built entirely on the `http_throughput` Criterion bench. To validate it, real standalone `cosmian_kms`/`ckms` binaries were built and a new throwaway load tool (`crate/clients/`

`client/examples/multi_server_load_test.rs`, **temporary, not committed**) was written to drive **continuous** concurrent load against real HTTP server processes — no Criterion involved. The result did not confirm the criterion numbers; it contradicted them:

Configuration	Criterion bench	Real continuous load (this tool)	Ratio
1 process, 4 workers	11,479 signs/sec	**25,820 signs/sec**	2.25×
1 process, 8 workers	13,455 signs/sec	**41,239 signs/sec**	3.06×
2 processes × 4 workers (combined, same machine)	*n/a*	**40,922 signs/sec**	*n/a*

Root cause: the Criterion bench dispatches 16 requests, `join_all`s (barrier — waits for the slowest straggler), *then* dispatches the next 16 — repeated task-spawn and synchronization-stall overhead on every round. The new tool spawns 16 workers once, each looping continuously with no barrier between requests — the same methodology the repo's own `ckms bench --load` tool already uses (`documentation/docs/benchmarks/ckms_bench/report.md`: "N concurrent async tasks send pre-serialised requests in tight loops"). **The lesson: the purpose-built repo benchmark (`ckms bench --load`) is the right tool for throughput sizing; the Criterion `http_throughput` bench (designed to prove CPU-scaling, not measure absolute throughput ceilings) understates sustained capacity substantially.**

The "many small processes beat one big process" conclusion above is also revised. 1 process × 8 workers (41,239) and 2 processes × 4 workers combined (40,922) are statistically the same — process count barely matters *on one shared machine*, because both configurations compete for the same finite hardware. The sub-linear per-core curve driving that earlier conclusion was substantially a Criterion measurement artifact, not a real architectural ceiling in the KMS server itself. What *does* scale close to linearly is **separate physical servers** — each with its own independent hardware and no shared bottleneck, since `Sign` still has zero DB writes.

Corrected server-count estimate, using the real solo-4-worker figure (not the criterion one) translated to the i5-8400H via the same verified PassMark single-thread ratio (0.75, HT disabled per your security requirement — 4 real cores only):

Per i5-8400H (4 real cores), translated from real measurement	**19,380 signs/sec** (vs. 8,617 from the criterion-based estimate — 2.25× higher)
Per server, if 8 stated CPUs = 2 independent 4-core groups	~38,760 signs/sec
Servers needed for 136,000 signs/sec	**4** (down from the earlier estimate of 8)

Remaining caveats, stated plainly: - This sandbox is **Debian 13**, not the **Ubuntu Server** the target cluster runs — both are Linux/glibc, so the relative comparison should hold, but this is one more untranslated variable. - The CPU translation is still analytical (verified PassMark ratio, not a measurement on real i5-8400H hardware). - The "2 independent 4-core groups per server" assumption is **unverified** — this sandbox has no CPU pinning/isolation tooling to confirm 2 processes on genuinely non-overlapping physical cores would scale additively rather than contend; the same-machine test here necessarily shared one pool. If a real server's 8 CPUs don't decompose into 2 truly independent groups, the per-server figure could be lower. - **The only way to remove all three caveats at once: run `ckms bench --load` (or this session's `multi_server_load_test` tool) directly on the actual Ubuntu Server / i5-8400H target hardware.** Everything above is a defensible bound, not a substitute for that.

⚠️ Second correction: all of the above was measured over plain HTTP, not HTTPS

Every diagram and the flow matrix (§0c) document `HTTPS/TLS, TCP 9998` as the real production protocol. The real-load experiment above used plain `http://` for convenience. That's a real gap — TLS handshake and per-record encryption cost CPU too. Re-ran the same 3 configurations with the repo's own test certificate (`test_data/certificates/client_server/server/kmserver.acme.com.{crt,key}`) and

`--tls-cert-file`/`--tls-key-file`, connecting over real HTTPS with `accept_invalid_certs` (same pattern as this repo's own `test_data/aws_xks/ckms_cli.toml`, since the test cert's CN doesn't cover `localhost`):

Configuration	HTTP (previous)	HTTPS (real)	TLS overhead
1 process, 4 workers (solo)	25,820	25,023	3.1%
1 process, 8 workers (solo)	41,239	32,487	21.2%
2 processes × 4 workers (combined, same machine)	40,922	**22,894**	**44.1%**

TLS overhead is not a fixed tax — it compounds under contention. At low load (solo, 4 workers) it's nearly free (3%). At higher load it grows (21% solo at 8 workers), and it's worst when multiple TLS-terminating processes share the same CPU pool (44% for 2×4 combined) — TLS handshake/record encryption competes for the same cores as the signing work itself, and that competition gets worse, not better, as more independent processes each doing their own TLS termination pile onto shared hardware.

This also revises the earlier "process count barely matters" finding (from the HTTP-only experiment). Under HTTPS, 2 processes × 4 workers (22,894) is meaningfully *worse* than 1 process × 8 workers (32,487) — a ~30% gap that barely existed under plain HTTP. Process-count choice is not free once TLS is in the picture.

Revised server-count estimate, using the real HTTPS solo-4-worker figure (the least contended, most conservative-to-optimistic data point) translated to the i5-8400H:

Model	Per-server estimate	Servers for 136,000 signs/sec (failover pool)
Optimistic: 2 independent, non-contending 4-core groups	~37,565 signs/sec	4
Realistic: measured 2-process contention factored in	~17,184 signs/sec	**8**

The realistic figure lands at **8 servers** — coincidentally matching the very first (differently-reasoned, criterion-based) estimate from earlier in this section. That's not validation of the original reasoning, which was independently wrong (§6b's first correction); it means two separate corrections (real-load methodology, then real TLS overhead) moved the estimate in opposite directions and landed nearby.

Full cluster sizing (not just the failover pool)

The "8 servers" above is **the combined output of 2 surviving regions**, not the whole cluster. Translating to the actual 3-region topology, using the realistic (contention-adjusted) per-server figure of **17,184 signs/sec**:

	Calculation	Result
Servers per region	`⌈68,000 / 17,184⌉`	**4**
Total cluster (3 regions)	`4 × 3`	**12 servers**
Failover capacity check (2 regions × 4 servers)	`8 × 17,184`	137,472 signs/sec
Margin over the 136,000 target	`(137,472 − 136,000) / 136,000`	**1.1%**

⚠️ Worst-case scenarios

1. Worst case *within* the design envelope (1 region down — this is what N+1 was built for). The margin above is **1.1%** — essentially zero. This sizing was derived from the most conservative measured figure already (HTTPS, 2-process contention), so there is very little room left to absorb anything further: FIPS-mode overhead (still unmeasured — §7), a real i5-8400H performing even slightly below the PassMark-translated estimate, or network jitter during failover. **Recommendation:**

provision 5 servers/region (15 total) instead of 4/region (12 total) — failover capacity becomes 171,840 signs/sec, a **26.4% margin**, which is a real safety buffer rather than a rounding error.

2. Worst case: 1 region down *and* a server also lost in a surviving region (e.g. routine maintenance or a hardware fault coinciding with the regional outage — not a contrived scenario). At the current 4-servers/region sizing: `(4 + 3) × 17,184 = 120,288 signs/sec` — **only 88.4% of target, a real breach.** This is the concrete, quantified case for the 5-servers/region recommendation above: at 5/region, the same double-fault gives `(5 + 4) × 17,184 = 154,656`, still comfortably over target.

3. Worst case *beyond* the design envelope (2 regions down simultaneously). Explicitly out of scope per the earlier decision (§1 — "single-node-loss is sufficient," 5 regions would be needed to survive 2 simultaneous losses, which was declined). Quantified here for completeness: only 1 region survives, `4 × 17,184 = 68,736 signs/sec` — **50.5% of target.** Worth noting what does *not* break in this scenario: unlike the Galera/quorum model considered and rejected earlier (§4), PGD does not require a majority to keep serving — the 1 surviving region's database stays fully writable. The failure here is pure **capacity**, not availability: signing keeps working, just at half the required rate, until a second region recovers.

---

7. Sizing the exact process count — ⚠️ INCOMPLETE, blocked by environment

Goal: `processes_needed = target_throughput_per_region × p99_single_request_latency` (Little's Law — each process holds exactly 1 request in flight, per §6).

What was built: `crate/test_kms_server/tests/tmp_latency_check.rs` (temporary, **not committed**) — starts an in-process FIPS-mode KMS, creates an Ed25519 key pair, runs 30 warm-up + 300 **sequential** (non-concurrent) `Sign` calls, and reports mean/p50/p95/p99 latency plus the implied per-process signs/sec.

What happened: the test compiled successfully but **failed to start the server in FIPS mode** in this sandbox:

```
Error starting the KMS server: ServerError("OpenSSL: unable to load the required provider:
... filename(/usr/lib/x86_64-linux-gnu/ossl-modules/fips.so): ... No such file or directory ...")
```

`crate/crypto/build.rs` fell back to dynamic-linking the system OpenSSL earlier in the session ("Static OpenSSL libs not found… falling back to dynamic linking"), and this sandbox's system OpenSSL has no FIPS provider module installed. This is an environment limitation, not a code bug.

To finish this (next steps): 1. Run `cargo test --test tmp_latency_check -p test_kms_server -- --nocapture` in an environment with the OpenSSL 3.6.2 FIPS provider actually available (real CI runner, or a machine with `fips.so` installed/built per `crate/crypto/build.rs`). 2. Take the reported p99 latency and compute `processes_per_region = ceil(68,000 × p99_seconds)` (68,000/sec = the N+1 per-region failover load from §6), then add operational margin. 3. Delete `crate/test_kms_server/tests/tmp_latency_check.rs` once the number is captured — it's a throwaway measurement, not meant to be committed.

---

8. Session artifacts (current working-tree state)

```
M  crate/test_kms_server/benches/http_throughput.rs        (added eddsa_sign case — now RUN, see
§6b; keep, clean)
?? crate/test_kms_server/tests/tmp_latency_check.rs        (temporary — delete after use, see §7)
?? crate/clients/client/examples/multi_server_load_test.rs (temporary — delete after use, see §6b
correction)
```

`target/criterion/kms_bench/{ecdsa_sign,eddsa_sign}/` also now contains the full Criterion HTML report (regression analysis, violin plots) from the §6b run — not committed, local only. `target/`

`release/{cosmian_kms,ckms,examples/multi_server_load_test}` are also now built locally (not committed — build artifacts).

---

9. Open items requiring follow-up before this plan can be executed

1. ~~**Proteccio 3-site key-cloning ceremony**~~ — **RESOLVED**: will be performed via the Eviden HSM admin tool (vendor-native cloning, outside KMS software).
2. **Exact PKCS#11 process count** — blocked on a real FIPS-capable latency measurement (§7).
3. **EDB PGD commercial subscription** — new procurement dependency introduced by the §4 multi-master pivot; same category of external blocker as item 1.
4. **CAS/version-column schema change** — a real code change to `crate/server_database` (new columns + updated write paths across all mutating queries) is now in scope. Design decided (§4: version column, explicit KMIP conflict error on stale write, no silent auto-retry) but **not yet implemented**.
5. ~~**Partition/fencing policy**~~ — **RESOLVED**: Quorum Commit (2-of-3 majority) automatically fences an isolated minority region to read-only; no custom tooling needed (§4).
6. ~~**PGD Proxy vs. PgBouncer-only**~~ — **RESOLVED**: PGD Proxy evaluated and explicitly not adopted (its default write-leader routing model conflicts with the true-multi-master requirement); PgBouncer-only, as already decided (§4).
7. **Admin Server implementation** — the HA-pair/VIP/bastion topology is decided (§1) but not yet built: 2 new physical/virtual nodes per region (6 total), their own Keepalived config, and the per-region bastion/jump-host infrastructure are all still to be provisioned.

---

10. Bibliography

In-repo sources are cited inline throughout this report as `` `path/to/file.rs:line` `` — that convention is used exhaustively (crate source, `documentation/docs/`, `test_data/`) and is not repeated here. This section lists the **external web sources** verified this session, grouped by topic, so every non-codebase claim can be independently checked.

Eviden Trustway IP Protect (§0, §1, §0c)

- Eviden — "Eviden receives ANSSI standard qualification for its network security solution"
- Euronext — "Eviden receives ANSSI standard qualification for its network security solution"
- Eviden — "Eviden supports post-quantum algorithms with its network security solution Trustway IP Protect"
- The Free Library — "Eviden secures ANSSI qualification for Trustway IP Protect VPN"

Trustway Proteccio NetHSM web admin console (§1, §0c)

- Encryption Consulting — Atos Trustway Proteccio NetHSM integration guide (PDF)
- Encryption Consulting — Atos Trustway Proteccio integration guide
- NATO — Trustway Proteccio NetHSM data sheet
- Keyfactor Docs — Configuring a TrustWay Proteccio NetHSM

EDB Postgres Distributed (PGD) — architecture, commit scopes, licensing (§4)

- EDB Docs — PGD Eager Replication
- EDB Docs — PGD Durability and Sync Modes
- EDB Docs — PGD Commit At Most Once (CAMO)
- EDB Docs — PGD documentation home
- EDB Docs — PGD Deployment Patterns
- EDB Docs — PGD Cluster Maintenance
- The Tech Bag — EDB Postgres Distributed: Active-Active, Five-Nines Postgres
- PostgreSQL.org — PostgreSQL License

- EDB — EDB Postgres Subscriptions (PDF)
- EDB Store — store.enterprisedb.com
- PR Newswire — "EDB Releases PGD 6.4 with Quorum Commit, Bringing True Distributed Consistency to Mission-Critical Postgres"
- Patroni Docs — Patroni: A Template for PostgreSQL HA
- GitHub — EnterpriseDB/edb-patroni docs

PGD Proxy write-leader routing (§4)

- EDB Docs — PGD Proxy overview and configuration
- EDB Docs — Read-only routing with PGD Proxy
- EDB Docs — PGD Proxy cluster/node configuration
- EDB Docs — PGD 5.8 routing functions reference

Patroni/etcd vs. PGD comparison, general HA background (§4)

- oneuptime.com — How to Set Up PostgreSQL with Patroni for High Availability
- markaicode.com — PostgreSQL High Availability Architecture with Patroni and etcd
- Medium (@uniquebiwas) — Production-Ready PostgreSQL High Availability
- computingforgeeks.com — PostgreSQL High Availability Cluster with Patroni, etcd, HAProxy
- beebeeb.io — PostgreSQL High Availability With Patroni, Explained

Hardware — CPU specifications and relative performance (§6b)

- NotebookCheck — Intel Core i7-1265U Processor — Benchmarks and Specs
- CPUTronic — Intel Core i7-1265U
- pchardware.org — Intel Core i7-1265U
- CPU-Boss — Intel Core i7-1265U benchmarks and specifications
- PassMark CPUBenchmark — Intel i5-8400H vs Intel i7-11800H
- technical.city — Core i5-8400H vs Core i7-1265U
- CPU-Monkey — Intel Core i7-1265U vs Intel Core i5-8400H
- PassMark CPUBenchmark — Intel Core i7-1265U vs Intel Core i7-1365U
- PassMark CPUBenchmark — Single-thread rankings, page 2
- PassMark CPUBenchmark — Intel i5-8400 vs Intel i5-8400H
- PassMark CPUBenchmark — Intel i5-8400H vs Intel i5-8300H

> **Note.** One early web search for base i5-8400H core/thread/clock specs returned a synthesized answer without a source URL in the tool response — those specific figures (4 cores/8 threads/2.5GHz) are corroborated by the PassMark comparison links above, which cover the same CPU, so no fact in this report rests solely on an uncited source.